
Week 3 — In-Class Lab

Theme: Converting SQL into REST and Creating Views & Triggers on Supabase

This hands-on lab will introduce you to PostgREST, the REST interface that powers Supabase's database API. You'll learn how to: - Map SQL queries to HTTP GET requests, - Apply filters using query parameters, - Create **views** that encapsulate reusable queries, - Write **triggers** that enforce business logic or automatically log changes.

Everything will run in Supabase (or through Postman/cURL) using your project's REST URL and anon key.

Setup (5 min)

1. **Find your REST URL and API key:** In the Supabase dashboard go to Project → Settings → API and copy the REST URL (e.g. `https://<project-ref>.supabase.co/rest/v1`) and your anon key.
2. **Choose a tool:** Use either:
 - Supabase API Explorer (built into the dashboard),
 - `curl` from a terminal,
 - Postman (create a new request).
3. **Headers for every request:**

```
apikey: <your anon key>
Authorization: Bearer <your anon key>
Accept: application/json
```

For POST requests also include: Content-Type: application/json.

Assume your PGExercises tables (facilities, members, bookings) are already loaded in the public schema.

Activity A – Convert SQL to REST (25 min)

For each query: 1. Write the SQL. 2. Translate it into a PostgREST URL. 3. Execute the request and verify JSON matches SQL results.

Example Queries: - Query A1 – Active members with a surname. - Query A2 – Facilities costing more than £5. - Query A3 – Bookings in September 2012 with 2+ slots. - Query A4 – Facilities starting with "Tennis."

Deliverable A - Provide the original SQL and the REST URL. - Paste your curl/Postman call (redact your key). - Include a short sample JSON result. - Write a short explanation of how you mapped SQL → REST parameters.

Activity B – Create a View & Trigger (20 min)

Views let you expose complex queries as reusable, secure endpoints. Triggers allow automation (e.g., audit logging).

B1. Create a View (SQL editor)

Create a view of facilities with their total bookings:

```
CREATE OR REPLACE VIEW facility_bookings AS
SELECT f.facid, f.name,
       COUNT(b.bookid) AS total_bookings,
       COALESCE(SUM(b.slots),0) AS total_slots
FROM facilities f
LEFT JOIN bookings b ON f.facid = b.facid
GROUP BY f.facid, f.name;
```

Now expose the view through REST:

GET {REST_URL}/facility_bookings?select=facid,name,total_bookings,total_slots

B2. Create a Trigger (SQL editor)

Add an audit log for new member insertions:

```
CREATE TABLE IF NOT EXISTS member_audit (
  audit_id serial PRIMARY KEY,
  memid int,
  action text,
  action_time timestamptz DEFAULT now()
);

CREATE OR REPLACE FUNCTION log_new_member() RETURNS trigger AS $$
BEGIN
  INSERT INTO member_audit(memid, action)
  VALUES (NEW.memid, 'INSERT');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_member_insert
AFTER INSERT ON members
FOR EACH ROW
EXECUTE FUNCTION log_new_member();
```

Test the trigger:

```
INSERT INTO members (memid, surname, firstname, joindate)
VALUES (100, 'Test', 'User', now());
```

```
SELECT * FROM member_audit ORDER BY audit_id DESC LIMIT 1;
```

Deliverable B

- Provide the SQL code for your **view** and **trigger**.
- Show how you queried the view via REST (include curl/Postman call + JSON output).
- Show the audit log entry created by the trigger.
- Write a short note about security considerations (e.g., views only expose needed columns; triggers run under table permissions).

Tips & Notes - If you get a 401/403, recheck your headers (apikey + bearer token). - Views must be created in `public` (or schema exposed in your Supabase API settings). - Remember: Triggers are powerful – use them for logging, enforcing rules, or automating tasks, not for heavy business logic.